Machine Learning: Evaluating Learning Algorithms

Dr. Oybek Eraliev,

Department of Computer Engineering

Inha University In Tashkent.

Email: oybekeraliev7@gmail.com

Content

- Introduction to Evaluation
- Performance Metrics
- Implementation of Performance Metrics

Introduction to Evaluation

- In machine learning, evaluating the performance of learning algorithms is a crucial step in understanding how well a model generalizes to unseen data.
- The goal is not only to minimize error on the training set but also to ensure the model performs well on new, unseen data.
- Effective evaluation helps identify the strengths, weaknesses, and potential improvements needed in a model.

Introduction to Evaluation

Importance of Evaluating Learning Algorithms

1. Generalization Ability: The ultimate goal of a machine learning model is to generalize well on unseen data. Evaluation helps assess whether a model trained on a given dataset will work effectively on new data. Overfitting (performing well on training data but poorly on test data) and underfitting (poor performance on both) are common issues that proper evaluation helps to identify.
2. Model Selection: Machine learning often involves selecting the best model from several options. Without an objective evaluation, it's difficult to determine which model performs best. Evaluation metrics guide the selection by providing a standardized way to compare models.

Introduction to Evaluation

Importance of Evaluating Learning Algorithms

3. Parameter Tuning: Machine learning models often have hyperparameters that need to be optimized. Evaluation helps assess the effect of different hyperparameter settings and fine-tune them for optimal performance.

4. Understanding Model Strengths and Weaknesses: Evaluation enables practitioners to understand in which scenarios their model excels and where it struggles. For example, certain models may work well in high-dimensional spaces but poorly with noisy data, and evaluation helps in identifying such characteristics.

Introduction to Evaluation

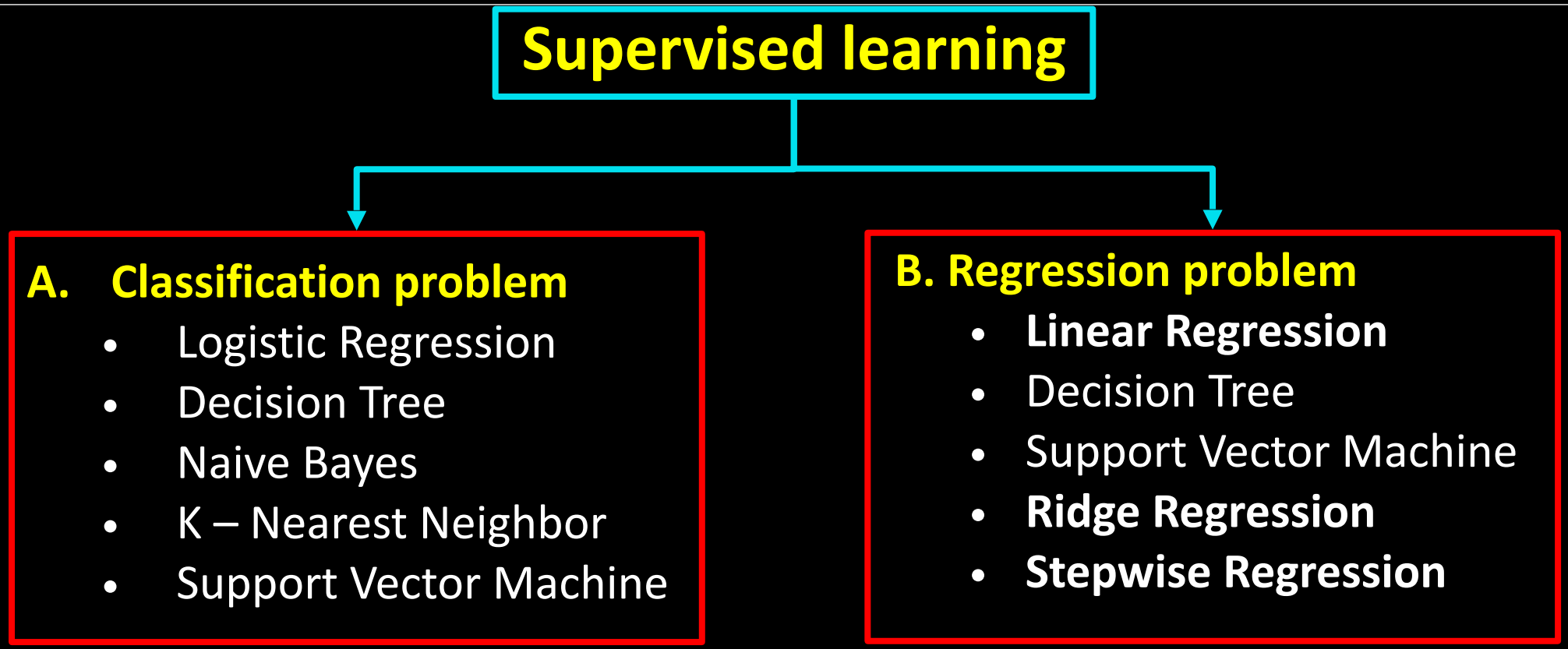
Importance of Evaluating Learning Algorithms

5. Benchmarking: To measure progress, models need to be evaluated against baselines and benchmarks. Evaluation metrics help compare the performance of the current model with previous models or other competitors' models.

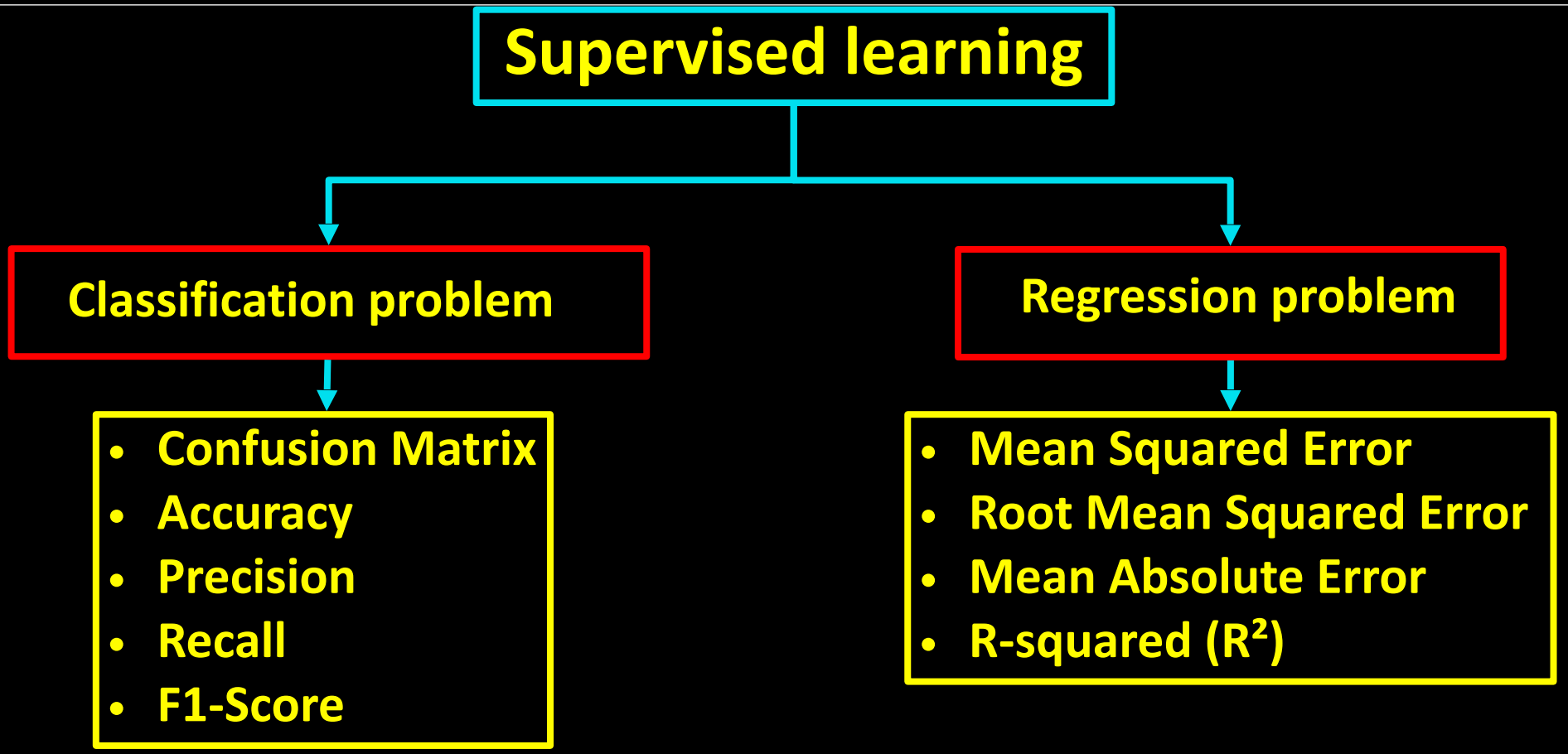
Content

- ~~Introduction to Evaluation~~
- Performance Metrics
- Implementation of Performance Metrics

Performance Metrics



Performance Metrics



Performance Metrics

Regression models

- Mean Squared Error
- Root Mean Squared Error
- Mean Absolute Error
- R-squared (R^2)

Mean Squared Error (MSE): Measures the average of the squares of the errors between predicted and actual values. It is sensitive to large errors.

$$MSE = \frac{1}{m} \cdot \sum_{i=1}^{m} (h_\theta(x)^{(i)} - y^{(i)})^2$$

Performance Metrics

Regression models

- Mean Squared Error
- Root Mean Squared Error
- Mean Absolute Error
- R-squared (R^2)

Root Mean Squared Error (RMSE): The square root of the MSE, giving error in the same units as the target variable.

$$MSE = \frac{1}{m} \cdot \sum_{i=1}^{m} (h_\theta(x)^{(i)} - y^{(i)})^2$$

$$RMSE = \sqrt{MSE}$$

Performance Metrics

Regression models

- Mean Squared Error
- Root Mean Squared Error
- Mean Absolute Error
- R-squared (R^2)

Mean Absolute Error (MAE): The average of the absolute differences between predicted and actual values. Less sensitive to large errors than MSE.

$$MAE = \frac{1}{m} \cdot \sum_{i=1}^{m} \left| h_\theta(x)^{(i)} - y^{(i)} \right|$$

Performance Metrics

Regression models

- Mean Squared Error
- Root Mean Squared Error
- Mean Absolute Error
- R-squared (R^2)

R-squared (R^2): Represents the proportion of the variance in the target variable explained by the model. A value close to 1 indicates a good fit.

$$R^2 = 1 - \frac{\sum_{i=1}^{m} (h_\theta(x)^{(i)} - y^{(i)})^2}{\sum_{i=1}^{m} (h_\theta(x)^{(i)} - y_{mean})^2}$$

$$y_{mean} = \sum_{i=1}^{m} y^{(i)}/m$$

Performance Metrics

Classification problem

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score

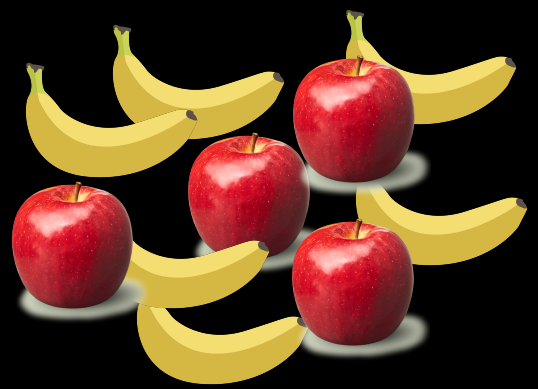
Confusion Matrix: A table that breaks down the predicted versus actual values, offering a detailed look at performance for each class (True Positives, False Positives, True Negatives, and False Negatives).

		Actuals: Positive	Actuals: Negative
Predicted	Positive	True Positives	False Positives
Predicted	Negative	False Negatives	True Negatives

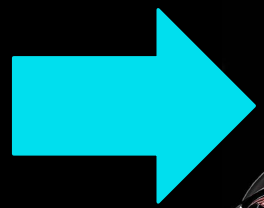
Performance Metrics

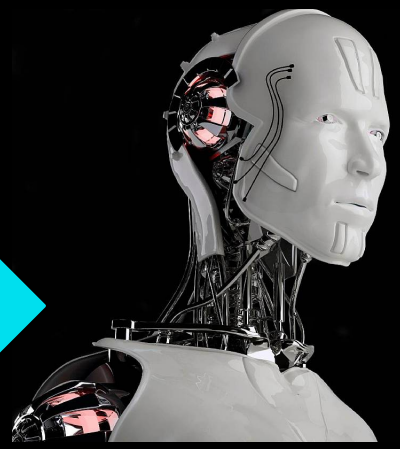
Classification problem

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score



Test set



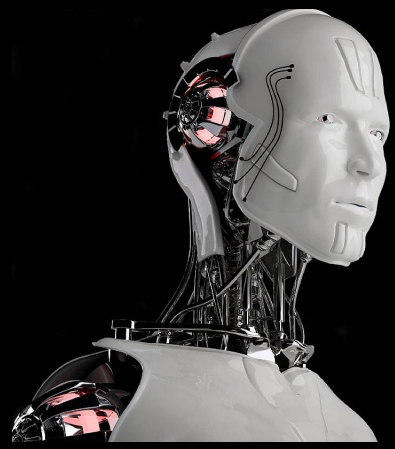


Model
(Algorithm)

Performance Metrics

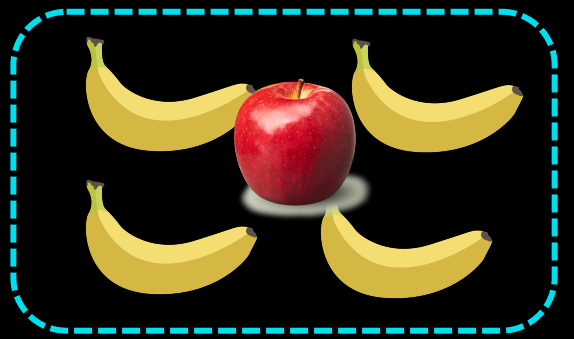
Classification problem

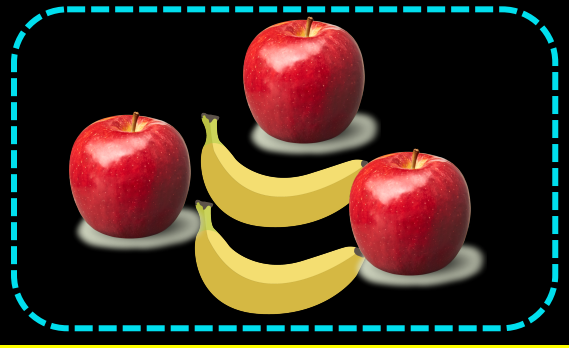
- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score



Model
(Algorithm)

Predictions

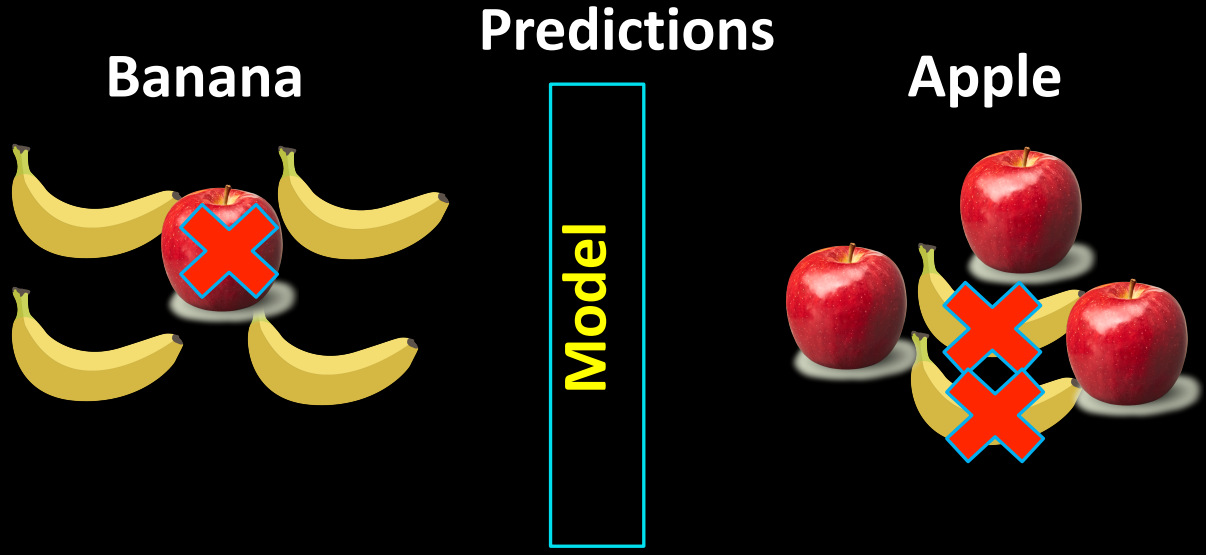




Performance Metrics

Classification problem

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score



Performance Metrics

Classification problem

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score

Banana = positive

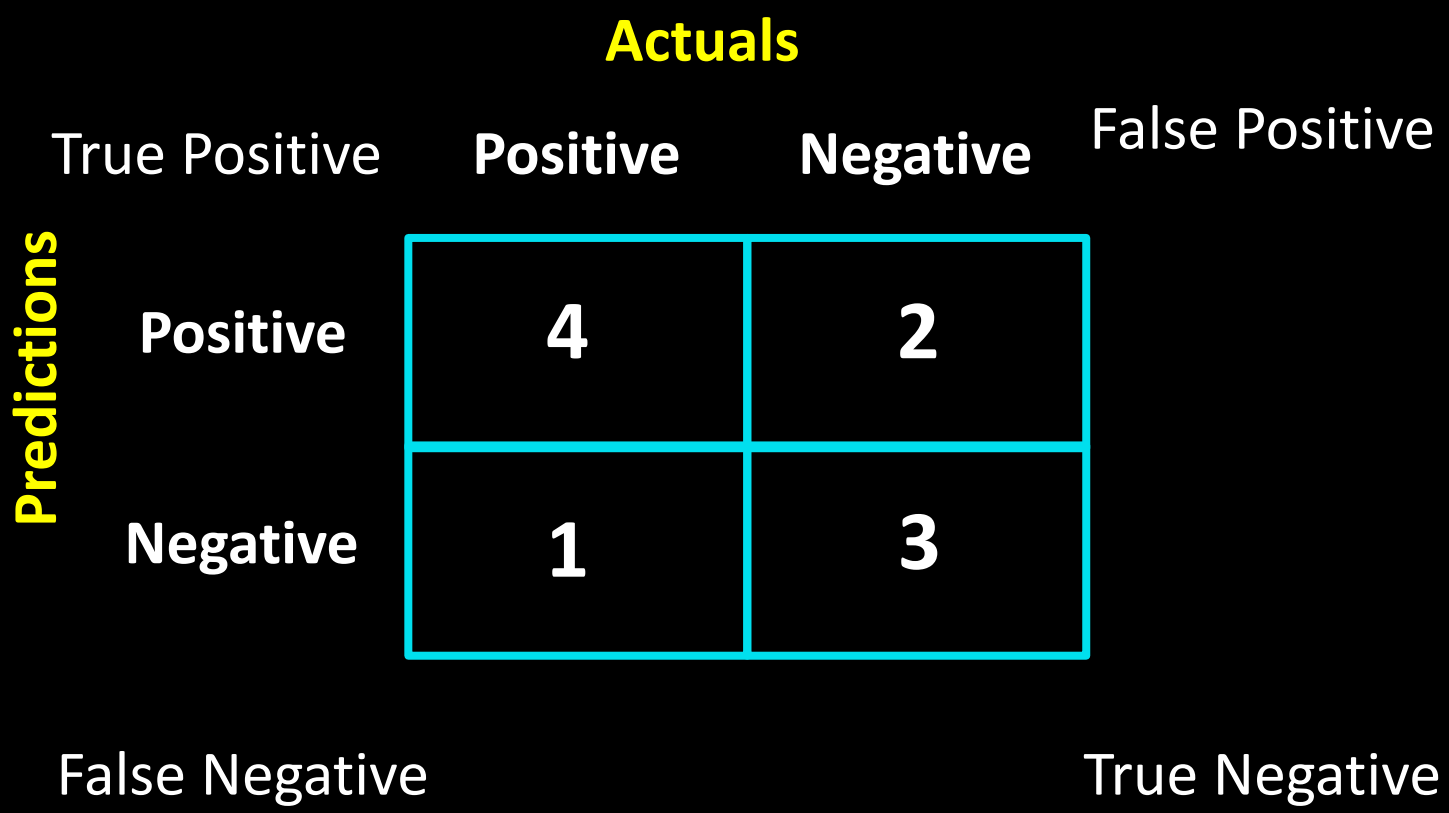
Apple = negative

Actuals \ Predictions	Banana	Apple
Banana	 4	 2
Apple	 1	 3

Performance Metrics

Classification problem

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score



Performance Metrics

Classification problem

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score

Accuracy: The proportion of correctly classified instances over the total number of instances.

$$Accuracy = \frac{Correct\ Predictions}{Total\ Predictions} \cdot 100\%$$

Actuals \ Predictions	Positive	Negative
Positive	True Positive **4**	False Positive **2**
Negative	False Negative **1**	True Negative **3**

$$Accuracy = \frac{4+3}{10} \cdot 100\% = 70\%$$

Performance Metrics

Classification problem

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score

Precision: The proportion of true positive instances among all instances predicted as positive. Precision is critical when false positives are costly.

$$Precision = \frac{True\ Positive}{True\ Positive + False\ Positive}$$

Actuals \ Predictions	Positive	Negative
Positive	True Positive 4	False Positive 2
Negative	False Negative 1	True Negative 3

$$\text{Precision} = \frac{4}{4+2} = 0.667$$

Performance Metrics

Classification problem

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score

It only looks at the predictions the model *made as positive* — it ignores the negative predictions entirely.

"Precision = Of all the alarms the model raised, how many were real?"

Precision matters most when a **false positive is expensive, embarrassing, or harmful** — i.e., wrongly flagging something as positive has real cost.

Spam email detection — A false positive means an important email (job offer, client message) gets buried in spam. Users trust the filter less if this happens often, so precision is prioritized over catching every spam email.

Cancer *screening* second-opinion / biopsy referral — (a nice contrast case) — Here you'd actually want high **recall**, not precision, because missing a real cancer (FN) is far worse than a false alarm. This is a great one to show students *why* precision isn't always the right metric — it depends on which error is more costly.

Performance Metrics

Classification problem

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score

Recall (Sensitivity): The proportion of true positive instances among all actual positive instances. It's useful when missing a positive instance (false negative) is costly.

$$Recall = \frac{True\ Positive}{True\ Positive + False\ Negative}$$

Actuals \ Predictions	Positive	Negative
Positive	True Positive **4**	False Positive **2**
Negative	False Negative **1**	True Negative **3**

$$\text{Recall} = \frac{4}{4+1} = 0.8$$

Performance Metrics

Classification problem

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score

Recall answers one question: "Of all the real positives out there, how many did the model actually catch?"

It only looks at the *actual* positive cases — it ignores how many false alarms (FP) were raised along the way.

Recall matters most when a **false negative is dangerous, costly, or irreversible** — i.e., missing a real positive case has serious consequences.

Cancer / disease screening — Missing an actual cancer patient (FN) can be fatal. A false alarm (FP) just leads to an extra test, which is annoying but not dangerous. So doctors accept lower precision for higher recall.

Airport security / weapon detection — Missing a real threat (FN) is catastrophic. Flagging an innocent bag for extra screening (FP) is just an inconvenience.

Performance Metrics

Classification problem

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score

F1 Score: The harmonic mean of precision and recall. It balances the trade-off between precision and recall and is useful when you need a single measure that accounts for both.

$$F1 = 2 \bullet \frac{Precision \bullet Recall}{Precision + Recall}$$

$$\text{Precision} = \frac{4}{4+2} = 0.667$$

$$\text{Recall} = \frac{4}{4+1} = 0.8$$

$$F1 = 2 \cdot \frac{0.667 \cdot 0.8}{0.667 + 0.8} = 0.73$$

Implementation of Performance Metrics

Regression models

```python
import numpy as np

# Predicted values (h(x)) and actual target values (y)
h_x = np.array([11, 19, 28, 35, 48, 60, 66, 78, 91, 99])
y = np.array([10, 20, 30, 40, 40, 60, 70, 80, 90, 100])

# Mean Squared Error (MSE)
mse = np.mean((y - h_x)**2)

# Root Mean Squared Error (RMSE)
rmse = np.sqrt(mse)

# Mean Absolute Error (MAE)
mae = np.mean(np.abs(y - h_x))

# R-squared (R²)
y_mean = np.mean(y)
ss_total = np.sum((y - y_mean)**2)
ss_residual = np.sum((y - h_x)**2)
r_squared = 1 - (ss_residual / ss_total)

# Print the results
print(f"MSE: {mse}")
print(f"RMSE: {rmse}")
print(f"MAE: {mae}")
print(f"R-squared: {r_squared}")
```

[1]

Implementation of Performance Metrics

Classification models

```python
import numpy as np

h_x_binary = np.array([0, 0, 0, 1, 1, 1, 0, 0, 0, 0])
y_binary = np.array([0, 0, 1, 1, 1, 0, 0, 0, 1, 0])

# Confusion matrix components
true_positive = np.sum((h_x_binary == 1) & (y_binary == 1))
true_negative = np.sum((h_x_binary == 0) & (y_binary == 0))
false_positive = np.sum((h_x_binary == 1) & (y_binary == 0))
false_negative = np.sum((h_x_binary == 0) & (y_binary == 1))

# Accuracy
accuracy = (true_positive + true_negative) / len(y_binary)

# Precision
precision = true_positive / (true_positive + false_positive) if (true_positive + false_positive) > 0 else 0

# Recall (Sensitivity)
recall = true_positive / (true_positive + false_negative) if (true_positive + false_negative) > 0 else 0

# F1 Score
f1_score = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0

# Confusion matrix
confusion_matrix = np.array([[true_negative, false_positive], [false_negative, true_positive]])

# Print the results
print(f"Predicted values: {h_x_binary}")
print(f"True values: {y_binary}")
print(f"Accuracy: {accuracy}")
print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"F1 Score: {f1_score}")
print("Confusion Matrix:")
print(confusion_matrix)
```

[6]




THANK YOU